Unit 7 Lecturecast – Artificial Neural Network (ANN)

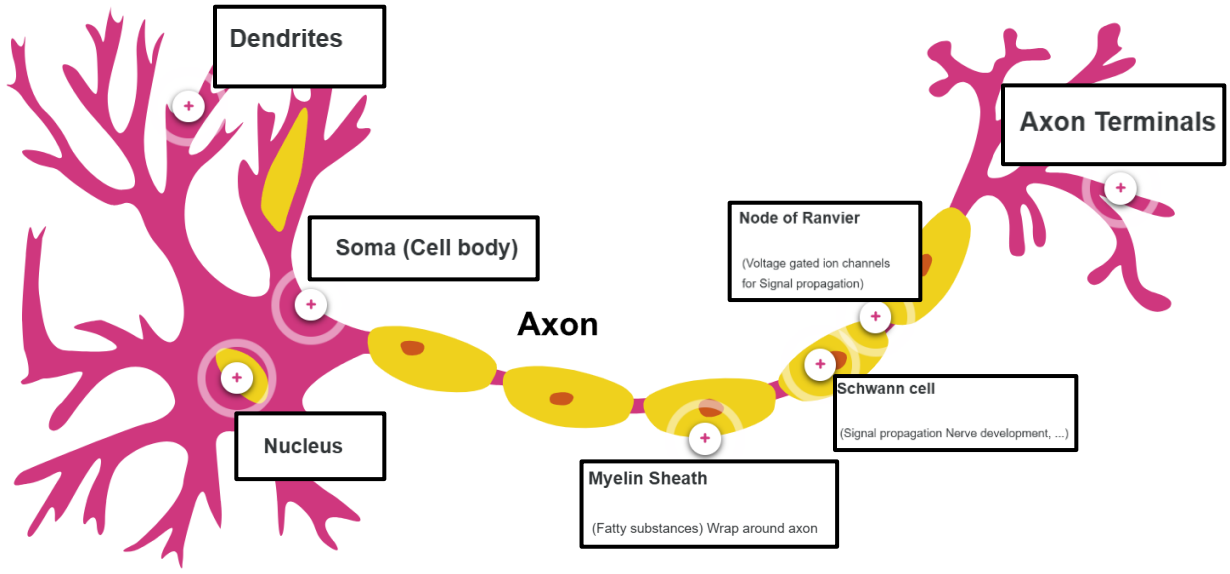
Lesson 1 - Biological Inspiration of ANN

- The design of ANN is motivated by the neural network of animal nervous system, particularly the brain.
- The brain is a living proof of fast and powerful fault-tolerant parallel processing
- Neurobiologists/Neuroscientists/Cognitive Scientists use ANN for modelling and interpretation of neurobiological phenomena and cognitive processes
    - Engineers look to Neuroscience for new ideas to solve complex problems with low power consumption

Lesson 2 - Human Brain

The human brain in particular is a living proof of a fast and powerful fault tolerant parallel processing system. The core central nervous system consists of Brain, Spinal Cord, and Peripheral nervous system (PNS) (sensory and motor nerve cells and networks). If compared with supercomputing of modern days which requires on average 7,630,848 cores, 5,087,232 GB and over 29kW of electricity, human brain can compute more complex work with minimal resources and energy. Some information on the human brain's ability: The Human brain consists of about 100 billion (101111) processing units (neurons) connected together in huge and complex network. The processing power/memory of human brain is originated from the interconnection among neurons, called synapses. On average, each neuron has 7k synaptic connection with other neurons. We will now review the parts responsible for the processing power in the human brain to understand how ANNs mimic their functionality.

Lesson 3 - Human Brain - Biological Neuron



Typical diameter of Soma: 3-18 μm

Longest axon for motor neurons that connects spinal cord to toes: ~1 m

Biological neurons are type of cells comprise of following parts:

- Dendrites: Dendrites that receives signals from other neurons or from the world if it is a sensory neuron.
- Soma: Soma is the cell body which has nucleus inside it. Soma process the neural signals.
- Axon: Axon is more or less the output end of the neuron which is connected with dendrites of other neuron or muscles or glands.

Lesson 4 - Human Brain - Types of Neurons

Sensory Neurons:

Dendrites are modified to become receptors (sensors) (also called Afferent neurons).

Motor Neurons:

Output neurons that stimulate muscles and glands to affect the outside world. Efferent neurons.

Interneurons:

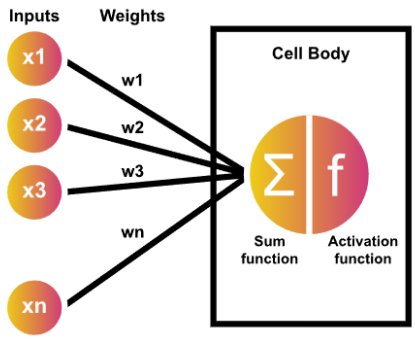
Connect sensory and motor neurons (cognitive processes).

Lesson 5 - Human Brain - Synapses

- Neurons communicate with one another through synapses
    - The axon terminal of a neuron impinges upon another neuron's dendrite
- Excitatory or inhibitory synapses
- Where learning occurs and knowledge stored (memory)
- Modelled as weights in artificial neurons

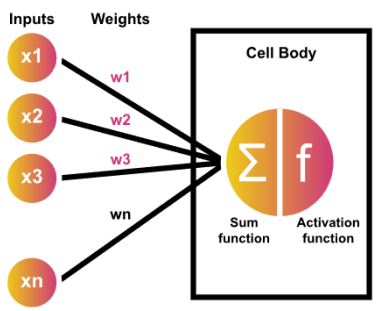
Synapses are the gap in between two neurons. A signal passes through the synapses via chemicals and the amount and type of chemical determine the amplitude of the signal. It is this synaptic clef where learning occurs and knowledge from the learning is kept in chemical secretion terms. The learning and memory aspect is modelled as weight in Artificial Neuron Networks.

Lesson 6 - Artificial Neuron/Perceptron



$$sum = \sum_{i=1}^{n} xi * wi$$

So what does an artificial neuron or perceptron do? Simply put, it calculates a “weighted sum” of its input, adds a bias (which is not shown in this representation), and then decides whether it should be “fired” or not. If fired, then it is classified as "yes" or "1" otherwise "no" or "0". The perceptron was invented by Frank Rosenblatt in the late '50s. The perceptron, therefore, works as a threshold function that provides a linear binary classifier.



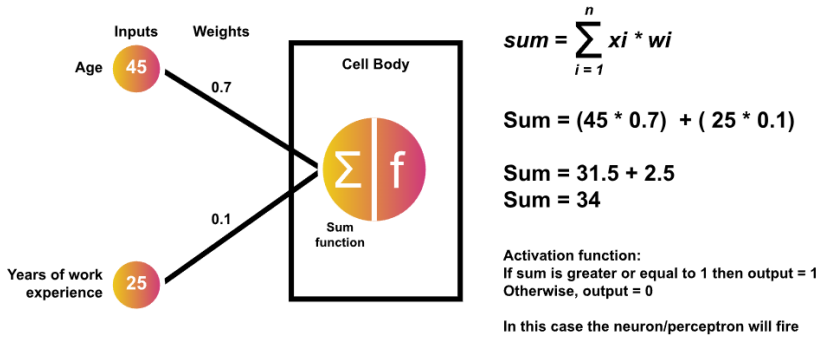
$$sum = \sum_{i=1}^{n} xi * wi$$

X1 * w1 + x2*w2 + x3 * w3

Let us look at it from a simple level.

Here the perceptron has *n* number of input and associated weights w. The perceptron first sums up the product of input and weights.

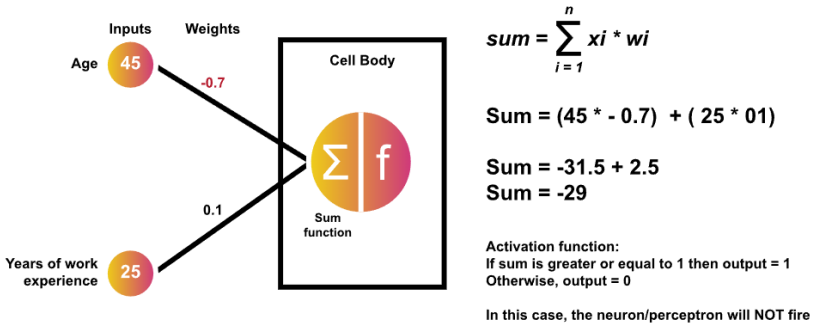
Example



Let us consider an example of this simple perceptron that has just two inputs, x1 and x2.

- Age is our first input (x1) with a value of 45.
- Years of work experience (x2) with a value of 25.
- Each of the inputs have an associate value of weights.

Example 2



- Age is our first input (x1) with a value of 45.
- Years of work experience (x2) with a value of 25.
- W1 = - 0.7•W2 = 0.1

Lesson 7 - Artificial Neuron/Perceptron

Formative Exercise 1 (See “Unit 7 Exercise Notes” document)

Lesson 8 - “AND” OPERATOR

For binary classification, we usually have two classes (yes-no or 1-0 type of output). Suppose you have this type of dataset where you apply the AND operator to classify (whether the player is a winner) each instance. Basically, each instance or record must fulfil a list of parameters or logic to be classified into either of the two classes.

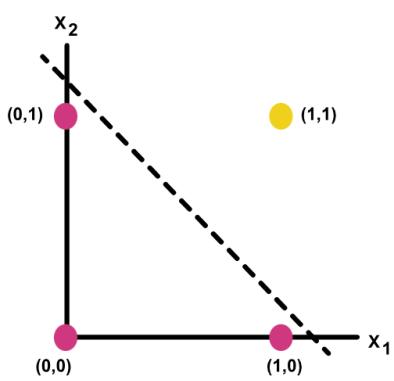
Player Name	Level 1 Clears?	Level 2 Clears?	Level 3 Clears?	Is Player Winner?
Player 1	Yes	Yes	No	?
Player 2	Yes	Yes	Yes	?
Player 3	Yes	Yes	No	?
Player 4	Yes	Yes	No	?
Player 5	No	Yes	Yes	?
Player 6	Yes	No	No	?
Player 7	No	Yes	Yes	?
Player 8	No	Yes	No	?
Player 9	Yes	Yes	No	?
Player 10	Yes	Yes	Yes	?

Formula	Output
=AND(TRUE,TRUE)	TRUE
=AND(TRUE,FALSE)	FALSE
=AND(FALSE,FALSE)	FALSE
=AND(FALSE,TRUE)	FALSE

Player Name	Level 1 Clears?	Level 2 Clears?	Level 3 Clears?	Is Player Winner?
Player 1	Yes	Yes	No	FALSE
Player 2	Yes	Yes	Yes	TRUE
Player 3	Yes	Yes	No	FALSE
Player 4	Yes	Yes	No	FALSE
Player 5	No	Yes	Yes	FALSE
Player 6	Yes	No	No	FALSE
Player 7	No	Yes	Yes	FALSE
Player 8	No	Yes	No	FALSE
Player 9	Yes	Yes	No	FALSE
Player 10	Yes	Yes	Yes	TRUE

x1	x2	Class
False	False	False
False	True	False
True	False	False
True	True	True

x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1



AND operator returns TRUE when logic 1 AND logic 2 are TRUE, in all other cases it is FALSE. This logic is represented by the numerical table and the data points on the graph. One linear line (black dotted line) can easily separate these two classes (pink and yellow points).

Lesson 9 - Weight Initialisation

Now we investigate how the adjustment of weights can help us to correctly classify the instances. For that, we initialise the weights w1 and w2 with 0. The sum function for all four instances was 0 and therefore none of the perceptrons fired. Then, we calculate the error against the original class and find that 3 out of 4 (75%) instances were correctly classified with this initial weight of 0. We calculate the error for each instance and use the sum of all errors (in this case 0 + 0 + 0 + 1 = 1) to update the weight for the next iteration.

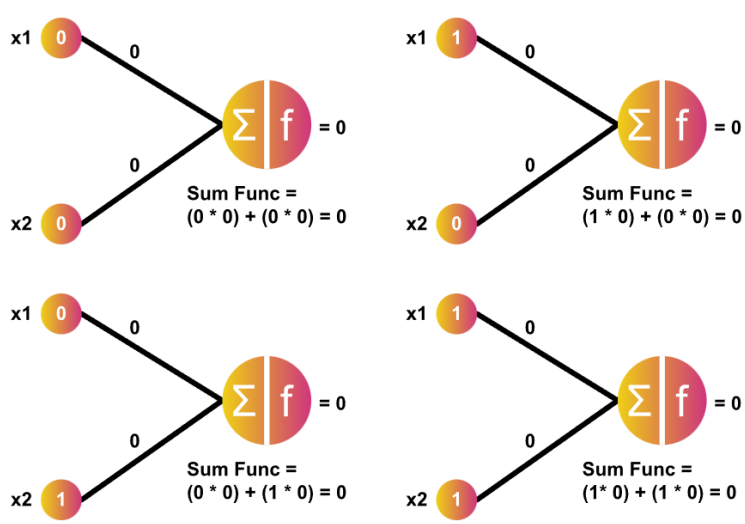
x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1

Class	Prediction	Error
0	0	0
0	0	0
0	0	0
1	0	1

RESULT

Error = abs(correct value – prediction value)

Accuracy = 75%



Activation function:

If Sum is Greater or equal to 1 then output = 1

Otherwise, output = 0

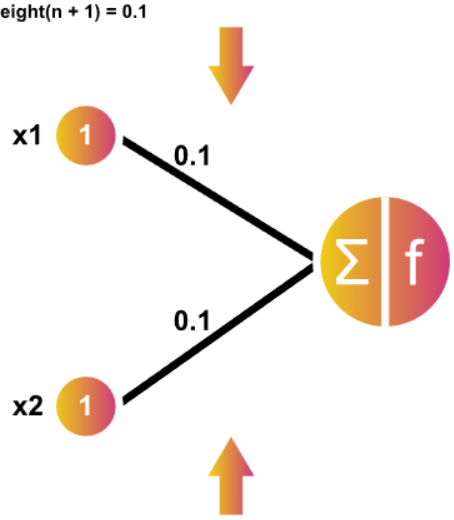
Lesson 10 - Weight Updating

Using weight(n), the weight used in previous iteration, and calculated error value, we assign a small positive number (in this case 0.1) as learning rate and go through next iteration.

x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1

Class	Prediction	Error
0	0	0
0	0	0
0	0	0
1	0	1

weight(n + 1) = weight(n) + (learning rate * input * error)
weight(n + 1) = 0 + (0.1 * 1 * 1)
weight(n + 1) = 0.1



weight(n + 1) = weight(n) + (learning rate * input * error)
weight(n + 1) = 0 + (0.1 * 1 * 1)
weight(n + 1) = 0.1

Although the value of sum function increased after the second iteration, the output is still 0.

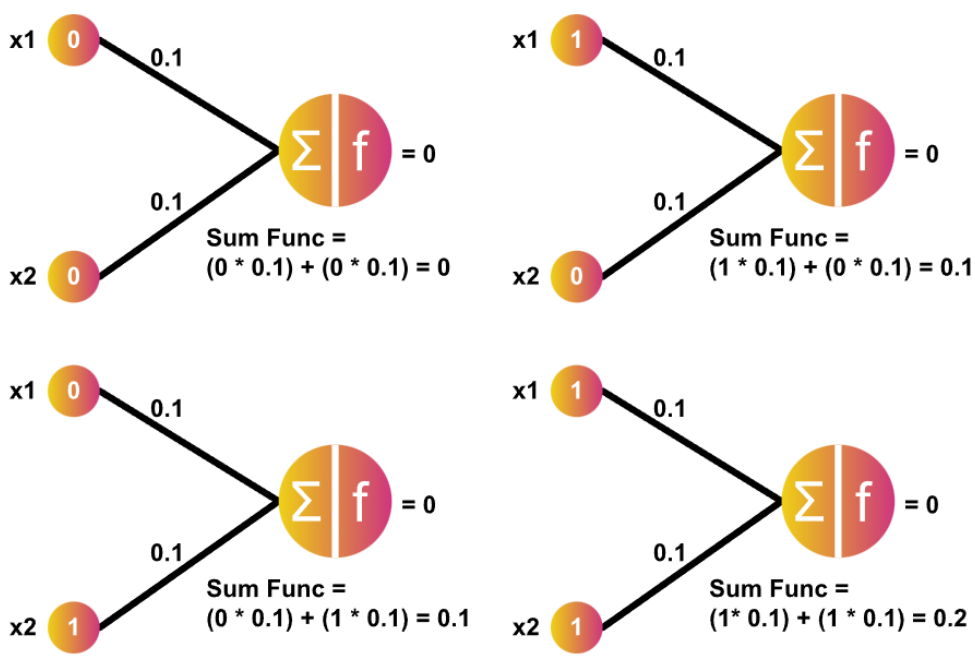
x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1

Class	Prediction	Error
0	0	0
0	0	0
0	0	0
1	0	1

RESULT

Error = abs(correct value – prediction value)

Accuracy = 75%

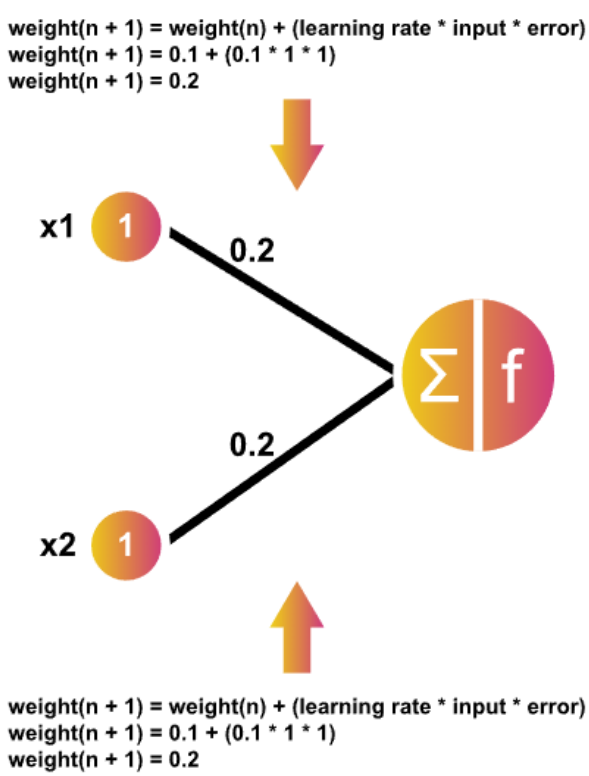


The weight is updated to 0.2, based on the previous iteration.

x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1

Class	Prediction	Error
0	0	0
0	0	0
0	0	0
1	0	1

Accuracy = 75%



After a few iterations, the weight becomes 0.5 and with that weight, sum functions classification becomes 100%.

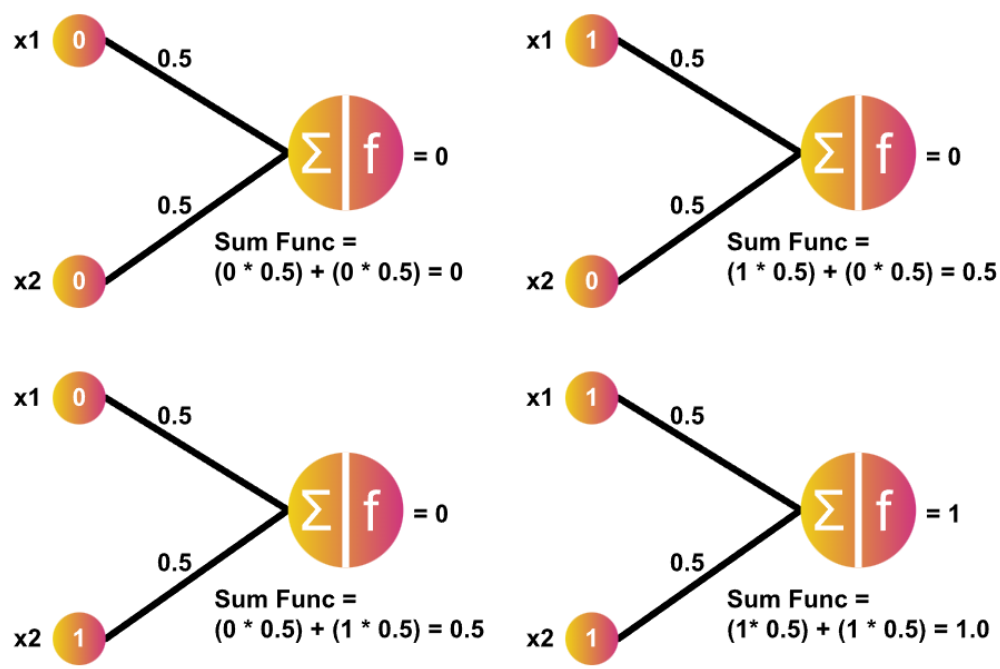
x1	x2	Class
0	0	0
0	1	0
1	0	0
1	1	1

Class	Prediction	Error
0	0	0
0	0	0
0	0	0
1	1	0

RESULT

Error = abs(correct value – prediction value)

Accuracy = 100%



Lesson 11 - Formative Exercise 2

You can pause the lecturecast at this point and run the second Python activity as mentioned in the module. (See “Unit 7 Exercise Notes” document)

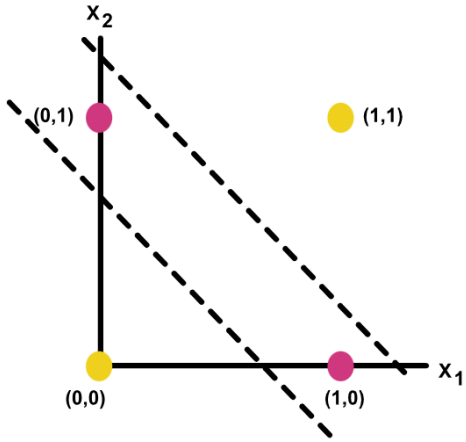
Lesson 12 - Formative Exercise 2

The XOR function performs what is called "exclusive OR". With two logical statements, XOR returns TRUE if either statement is TRUE, but returns FALSE if both statements are TRUE. If neither is TRUE, XOR also returns FALSE.

In such scenarios, a single linear line boundary as shown in the previous AND operator example will be incapable to classify these four instances. Therefore we need multiple or non-linear lines to classify accurately. In order to find these multiple lines, we need multiple layers of perceptrons.

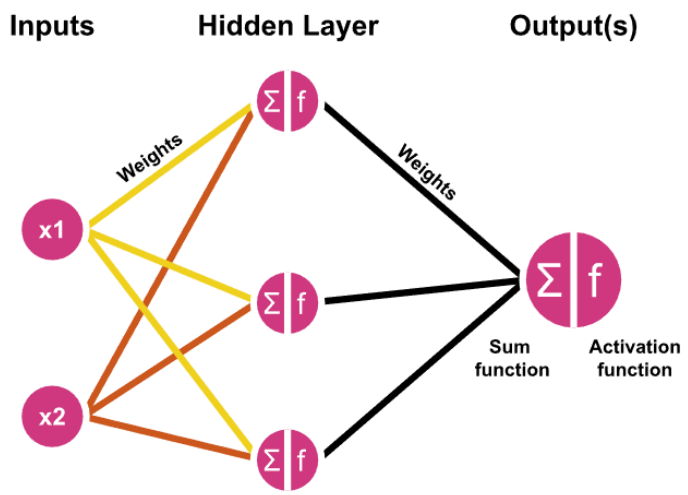
x1	x2	Class
False	False	False
False	True	True
True	False	True
True	True	False

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



Lesson 13 - Multi-Layer Perceptron (Multi-Layer Neural Networks)

So far, we have we have only the input and output layers. In this type of simple structure, there is no room for adjusting the structure (that is, the connection between neurons). Therefore, like a biological neuronal network, in the artificial neural network, several intermediary neurons are set to form a hidden layer.



Lesson 14 - Different Functions of Multi-layer Perceptron

- Activation functions
- Weight update (XOR operator)
- Error functions
- Gradient descent
- Backpropagation
- Implementation of a multi-layer perceptron from scratch using Python and Numpy

Lesson 15 - Different Functions of Multi-layer Perceptron

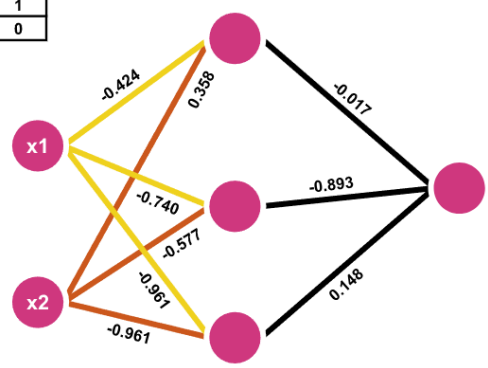
The weighted sum gives a value between – infinity to + infinity depending on the input value and weight applied. Based on the output value, if it is 0 or below, the neuron would not fire, and if 1 or above, it would fire. Such binary classification or step function has some drawbacks. What if you have multiple neurons are connected to give more classes? What will happen if more than 1 neuron is activated? In such a case, the output will be always "1" or "yes" and the classification will be one-sided. Instead of binary "1, 0", if we can say the neurons are activated 20% or 65% etc and if more than one neuron is activated, we would be able to find which neuron has the highest activation and the second highest etc. Indeed, there are possibilities that more than one neuron can have 100% activation but if we have an intermediary layer of neurons the possibility will be reduced.

Lesson 16 - Formative Exercise 3

You can pause the lecturecast at this point and run the last Python activity as mentioned in the module (the Sigmoid Function). (See “Unit 7 Exercise Notes” document)

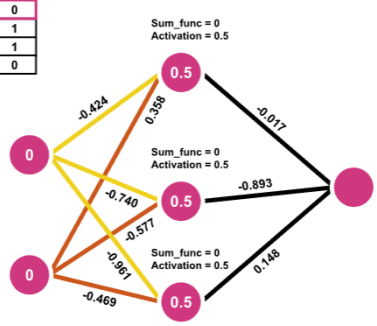
Lesson 17 - Input Layer to Hidden Layer

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



Now, let us consider an ANN that has multiple neurons as middle layer and Sigmoid Activation Function - how we can solve the XOR operator problem. We will now work through this example to review the correct process.

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

Sums for Inputs to Hidden Layer

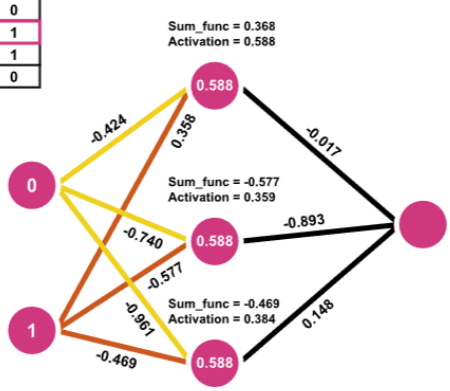
(0 * -0.424) + (0 * 0.358) = 0
(0 * -0.740) + (0 * -0.577) = 0
(0 * -0.961) + (0 * -0.469) = 0

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]:  def sigmoid(sum_func):
             return 1 / (1 + np.exp(-sum_func))
In [7]:  sigmoid(0)
Out[7]:  0.5
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

Sums for Inputs to Hidden Layer

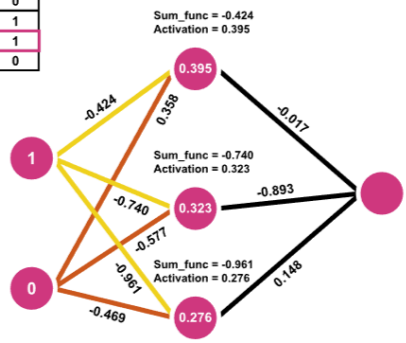
(0 * -0.424) + (1 * 0.358) = 0.358
(0 * -0.740) + (1 * -0.577) = -0.577
(0 * -0.961) + (1 * -0.469) = -0.469

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]:  def sigmoid(sum_func):
             return 1 / (1 + np.exp(-sum_func))
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

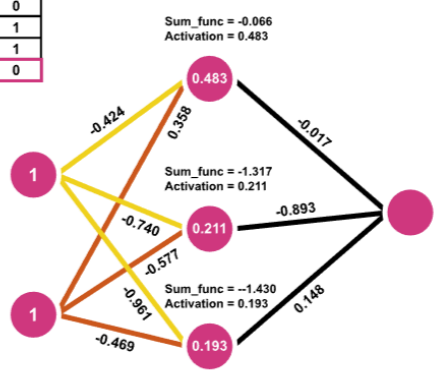
Sums for Inputs to Hidden Layer
(1 * -0.424) + (0 * 0.358 = -0.424
(1 * -0.740) + (0 * -0.577) = -0.740
(1 * -0.961) + (0 * -0.469) = -0.961

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]:  def sigmoid(sum_func):
             return 1 / (1 + np.exp(-sum_func))
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

Sums for Inputs to Hidden Layer
1 * (-0.424) + 1 * 0.358 = -0.066
1 * (-0.740) + 1 * (-0.577) = -1.317
1 * (-0.961) + 1 * (-0.469) = -1.430

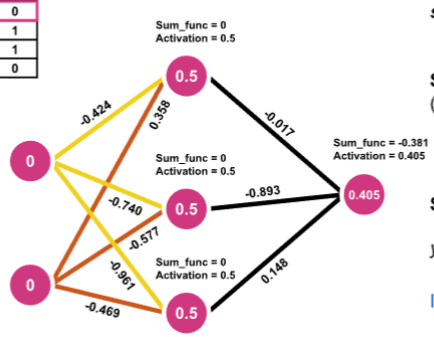
Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]:  def sigmoid(sum_func):
             return 1 / (1 + np.exp(-sum_func))
```

Lesson 18 - Hidden Layer to Output Layer

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

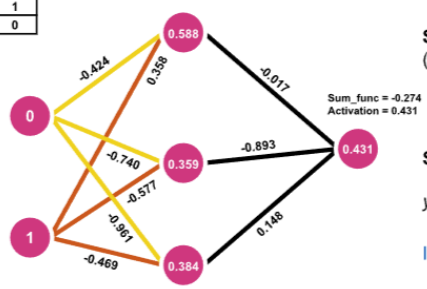
Sums for output layer
(0.5 * -0.017) + (0.5 * -0.893) + (0.5 * 0.148) = -0.381

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]:  def sigmoid(sum_func):
             return 1 / (1 + np.exp(-sum_func))
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

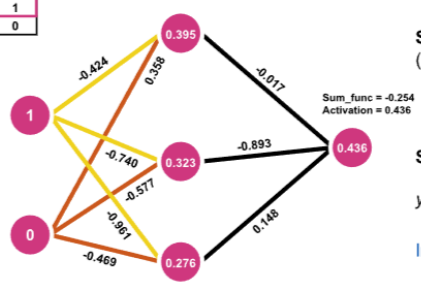
Sums for output layer
(0.589 * -0.017) + (0.360 * -0.893) + (0.385 * 0.148) = -0.274

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]: def sigmoid(sum_func):
        return 1 / (1 + np.exp(-sum_func))
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0



$$sum = \sum_{i=1}^{n} xi * wi$$

Sums for output layer
(0.395 * -0.017) + (0.323 * -0.893) + (0.277 * 0.148) = -0.254

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]: def sigmoid(sum_func):
        return 1 / (1 + np.exp(-sum_func))
```

x1	x2	Class
0	0	0
0	1	1
1	0	1
1	1	0

$$sum = \sum_{i=1}^{n} xi * wi$$

Sums for output layer
(0.483 * -0.017) + (0.211 * -0.893) + (0.193 * 0.148) = -0.168

Sigmoid Activation Function

$$y = \frac{1}{1 + e^{-x}}$$

```
In [2]: def sigmoid(sum_func):
        return 1 / (1 + np.exp(-sum_func))
```

Lesson 19 - XOR Operator – Error (Loss Function)

Error = correct (class) – prediction

x1	x2	Class	Prediction	Error
0	0	0	0.405	-0.405
0	1	1	0.431	0.569
1	0	1	0.436	0.564
1	1	0	0.458	-0.458

Average Error = abs(error) = 0.499

This is the output from a single iteration. You would need to make several iterations to reduce the error.

References

Sharma, A. (2017) Understanding Activation Functions in Neural Networks. Available from: **https://medium.com/the-theory-of-everything/understanding-activation-functions-in-neural-networks-9491262884e0** [Accessed 03 May2022]

Top 500 The List (n.d.) Supercomputer Fugaku. Available from: **https://www.top500.org/system/179807** [Accessed 03 May2022]

Mayo, M. (2017) Neural Network Foundations, Explained: Updating Weights with Gradient Descent & Backpropagation. Available from: https://www.kdnuggets.com/2017/10/neural-network-foundations-explained-gradient-descent.html [Accessed 03 May2022]

Math is fun (2020) Introduction to Derivatives. Available from: https://www.mathsisfun.com/calculus/derivatives-introduction.html [Accessed 03 May2022]